Abhiva
Practice • Improve • Perform

ASSESSMENT REPORT

Name : Vivek N P
Email : viveknp80@gmail.com
Assessment : JavaScript Assessment -REG - React JS - 04 May 2026
Completed On : 2026-07-01

TOTAL SCORE	QUESTIONS	TIME TAKEN	RANK
78.6%	3/5	74 min	#1/6

PERFORMANCE ANALYTICS

Coding Questions

Total : 5
Correct : 3
Incorrect : 2

DETAILED QUESTION REVIEW

Q1
Question : Predict the console output order for the following JavaScript code snippet. Your function `predictOrder1` should simulate the execution and return an array of strings representing the order of logs.

```javascript
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
console.log('D');
```

Your Answer : const readline = require('readline');

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function predictOrder1() {
  // your code here
  // The function should return an array of strings,
  // where each string is a log message in the order it would appear.
  // For example: ['A', 'D', 'C', 'B']

let order=[];
order.push('A');
order.push('D');
order.push('C');
order.push('B');
return order;
}

rl.on('line', (line) => {
  const result = predictOrder1();
  console.log(result.join(','));
  rl.close();
});
```

Correct Answer :

```
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function predictOrder1() {
  const logs = [];
  const microtasks = [];
  const macrotasks = [];

  logs.push('A');

  macrotasks.push('B'); // setTimeout callback

  microtasks.push('C'); // Promise.then callback

  logs.push('D');

  return [...logs, ...microtasks, ...macrotasks];
}
```

```
rl.on('line', (line) => {
   const result = predictOrder1();
   console.log(result.join(','));
   rl.close();
});
```

Points : 100/100
Status : Correct
Code Logic Score : 100.0
Space Complexity : O(1)
Time Complexity : O(1)

TEST CASE RESULTS

S.No	Input	Expected Output	Actual Output	Status
1	start	A,D,C,B	A,D,C,B	PASS
2	start	A,D,C,B	A,D,C,B	PASS
3	start	A,D,C,B	A,D,C,B	PASS
4	start	A,D,C,B	A,D,C,B	PASS
5	start	A,D,C,B	A,D,C,B	PASS
6	start	A,D,C,B	A,D,C,B	PASS
7	start	A,D,C,B	A,D,C,B	PASS
8	start	A,D,C,B	A,D,C,B	PASS
9	start	A,D,C,B	A,D,C,B	PASS
10	start	A,D,C,B	A,D,C,B	PASS
11	start	A,D,C,B	A,D,C,B	PASS
12	start	A,D,C,B	A,D,C,B	PASS
13	start	A,D,C,B	A,D,C,B	PASS
14	start	A,D,C,B	A,D,C,B	PASS
15	start	A,D,C,B	A,D,C,B	PASS

Q2

Question : Given an array of strings strs, group the anagrams together. You can return the answer in any order.
Your Answer : const readline = require('readline');

```
const rl = readline.createInterface({
   input: process.stdin,
```

```
  output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});

rl.on('close', () => {
  const strs = inputLines;
  const result = groupAnagrams(strs);
  result.forEach(group => {
    console.log(group.sort().join(' ')); // Sort each group for consistent output
  });
});

function groupAnagrams(strs) {

const map= new Map();
for (const word of strs){
const key=word.split('').sort().join('');
if (!map.has(key)){
map.set(key,[]);
}
map.get(key).push(word);
}
return Array.from(map.values());
}
```

Correct Answer : const readline = require('readline');

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});

rl.on('close', () => {
  const strs = inputLines;
  const result = groupAnagrams(strs);
  result.forEach(group => {
    console.log(group.sort().join(' ')); // Sort each group for consistent output
```

```
  });
});

function groupAnagrams(strs) {
  const map = new Map();

  for (const str of strs) {
    const sortedStr = str.split('').sort().join('');
    if (map.has(sortedStr)) {
      map.get(sortedStr).push(str);
    } else {
      map.set(sortedStr, [str]);
    }
  }

  return Array.from(map.values());
}
```

Points : 0/100
Status : **Incorrect**

TEST CASE RESULTS

Compilation error, No test cases were passed

Q3
Question : Given an array `nums` of size `n`, return the majority element. The majority element is the element that appears more than `n / 2` times. You may assume that the majority element always exists in the array.
Your Answer :

```
const readline = require('readline');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function majorityElement(nums) {
  // your code here
let candidate=null;
let count=0;
for(let num of nums){
if(count===0){
candidate=num;
}
if(num===candidate){
count++;
```

```
}else{
count--;
}
}
  return candidate;
}

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});

rl.on('close', () => {
  const nums = inputLines[0].split(' ').map(Number);
  const result = majorityElement(nums);
  console.log(result);
});
```
Correct Answer : const readline = require('readline');

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function majorityElement(nums) {
  let candidate = 0;
  let count = 0;

  for (let i = 0; i < nums.length; i++) {
      if (count === 0) {
          candidate = nums[i];
          count = 1;
      } else if (nums[i] === candidate) {
          count++;
      } else {
          count--;
      }
  }
  return candidate;
}

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});
```

```
rl.on('close', () => {
   const nums = inputLines[0].split(' ').map(Number);
   const result = majorityElement(nums);
   console.log(result);
});
```

Points : 100/100
Status : Correct
Code Logic Score : 95.0
Space Complexity : O(1)
Time Complexity : O(n)

TEST CASE RESULTS

S.No	Input	Expected Output	Actual Output	Status
1	3 2 3	3	3	PASS
2	2 2 1 1 1 2 2	2	2	PASS
3	1	1	1	PASS
4	7 7 7 7 7 7 7	7	7	PASS
5	1 2 1 2 1	1	1	PASS
6	6 5 5	5	5	PASS
7	10 9 10 9 10	10	10	PASS
8	1 1 2 2 1 1 2 2 1	1	1	PASS
9	5 5 5 1 1 1 5	5	5	PASS
10	3 3 3 3 2 2 2	3	3	PASS
11	1 2 3 4 5 5 5 5 5	5	5	PASS
12	1 1 1 1 1 2 3 4 5	1	1	PASS
13	10 20 10 20 10 20 10	10	10	PASS
14	1 1 1 1 1 1 1 1 1 1 2 3 4 5 6 7 8 9 10	1	1	PASS
15	100 100 100 100 100 100 100 100 100 100 1 2 3 4 5 6 7 8 9 10	100	100	PASS

Q4

Question : Write a function `mergeAndSumUnique` that accepts an arbitrary number of arrays of numbers using the rest parameter syntax. The function should merge all arrays, remove duplicate numbers, and then return the sum of the unique numbers. Use ES6 array methods like `Set` and `reduce`.

Your Answer : const mergeAndSumUnique = (...arrays) => {
 // your code here
 return [...new Set(arrays.flat())].reduce((sum,num)=> sum + num,0);
};

```
const readline = require('readline');
const rl = readline.createInterface({
 input: process.stdin,
 output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
 inputLines.push(line);
});

rl.on('close', () => {
 const arrays = inputLines.map(line => line.split(' ').map(Number));
 console.log(mergeAndSumUnique(...arrays));
});
```

Correct Answer : const mergeAndSumUnique = (...arrays) => {
 const allNumbers = [].concat(...arrays);
 const uniqueNumbers = new Set(allNumbers);
 return Array.from(uniqueNumbers).reduce((sum, num) => sum + num, 0);
};

```
const readline = require('readline');
const rl = readline.createInterface({
 input: process.stdin,
 output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
 inputLines.push(line);
});

rl.on('close', () => {
 const arrays = inputLines.map(line => line.split(' ').map(Number));
 console.log(mergeAndSumUnique(...arrays));
});
```

Points : 100/100
Status : **Correct**
Code Logic Score : 100.0
Space Complexity : O(N)

Time Complexity : O(N)

TEST CASE RESULTS

S.No	Input	Expected Output	Actual Output	Status
1	1 2 3 3 4 5 5 6 7	28	28	PASS
2	10 20 20 30 30 40	100	100	PASS
3	1 1 1 2 2 2	3	3	PASS
4	5 5 5	5	5	PASS
5	1 2 3 4 5	15	15	PASS
6	0 0 0	0	0	PASS
7	100 200 300 400	1000	1000	PASS
8	7 8 9 9 10 11	45	45	PASS
9	1 2 3 4 5 6 7 8 9 10	55	55	PASS
10	1 3 5 2 4 6	21	21	PASS
11	10 10 10 20 20 20 30 30 30	60	60	PASS
12	1 2 3 4 5 6 7 8 9 10	55	55	PASS
13	1 1 2 2 3 3	6	6	PASS
14	10 20 30 10 20 30	60	60	PASS
15	1 2 3 4 5	15	15	PASS

Q5

Question : Given an array of strings `strs`, group the anagrams together. You can return the answer in any order. An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Your Answer : const readline = require('readline');

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});

rl.on('close', () => {
  const n = parseInt(inputLines[0]);
  const strs = [];
  for (let i = 1; i <= n; i++) {
    strs.push(inputLines[i]);
  }

  const result = groupAnagrams(strs);
  // Sort groups and elements within groups for consistent output
  result.forEach(group => group.sort());
  result.sort((a, b) => a[0].localeCompare(b[0]));

  result.forEach(group => {
    console.log(group.join(' '));
  });
});

function groupAnagrams(strs) {
  // your code here
const map=new Map();
for(const str of strs){
const sorted = str.split('').sort().join('');
if(!map.has(sorted)){
map.set(sorted,[]);
}
map.get(sorted).push(str);
}
return Array.from(map.values());
}
```

Correct Answer : const readline = require('readline');

```
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

let inputLines = [];
rl.on('line', (line) => {
  inputLines.push(line);
});

rl.on('close', () => {
  const n = parseInt(inputLines[0]);
  const strs = [];
  for (let i = 1; i <= n; i++) {
    strs.push(inputLines[i]);
  }

  const result = groupAnagrams(strs);
  // Sort groups and elements within groups for consistent output
  result.forEach(group => group.sort());
  result.sort((a, b) => a[0].localeCompare(b[0]));

  result.forEach(group => {
    console.log(group.join(' '));
  });
});

function groupAnagrams(strs) {
  const anagramMap = new Map();

  for (const str of strs) {
    const sortedStr = str.split('').sort().join('');
    if (anagramMap.has(sortedStr)) {
      anagramMap.get(sortedStr).push(str);
    } else {
      anagramMap.set(sortedStr, [str]);
    }
  }

  return Array.from(anagramMap.values());
}
```

Points : 93/100

Status : **Incorrect**

Code Logic Score : 100.0
Space Complexity : O(N * L)
Time Complexity : O(N * L log L)

TEST CASE RESULTS

S.No	Input	Expected Output	Actual Output	Status
1	4 eat tea tan nat	eat tea nat tan	eat tea nat tan	PASS
2	3 hello world apple	apple hello world	apple hello world	PASS
3	1 a	a	a	PASS
4	0			PASS
5	2 a b	a b	a b	PASS
6	5 listen silent enlist cat act	act cat enlist listen silent	act cat enlist listen silent	PASS
7	3 rat tar art	art rat tar	art rat tar	PASS
8	4 flower flow wolf fowl	flow flower fowl wolf	flow fowl wolf flower	FAIL
9	2 abc bca	abc bca	abc bca	PASS
10	2 xyz yxz	xyz yxz	xyz yxz	PASS

11	5 stop pots tops spot post	post pots spot stop tops	post pots spot stop tops	**PASS**
12	3 aa a b	a aa b	a aa b	**PASS**
13	4 aba aab bba bab	aab aba bab bba	aab aba bab bba	**PASS**
14	2 apple banana	apple banana	apple banana	**PASS**
15	6 code doce ecod aced dace caed	aced caed dace code doce ecod	aced caed dace code doce ecod	**PASS**